

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Introducción a la Programación y Computación 2
Sección “A”
Ing. José Moisés Granados Guevara



MANUAL TÉCNICO

Proyecto 1

Jonathan Mauricio Castro Díaz

202431793

Quetzaltenango, 17 de septiembre de 2026

I. Descripción General

El sistema web 'Transportes del Norte' es una plataforma integral de gestión logística, operativa y contable desarrollada a la medida para una empresa líder en transporte extraurbano en Guatemala, bajo encargo de la firma de desarrollo 'Code 'n Bugs'.

La solución sustituye los controles dispersos y manuales por una arquitectura web cliente/servidor centralizada basada en estándares empresariales de Java EE (Jakarta Servlets y JSP). El sistema automatiza el ciclo comercial completo: desde la comercialización digital de boletos con selección gráfica de asientos y contratación personalizada de viajes privados, hasta la logística de despacho de unidades, liquidación de combustible por odómetro, registro de mantenimientos mecánicos y consolidación contable de ingresos, costos y ganancias netas por sucursal.

1. Objetivos

- Centralizar la administración de la flota vehicular, asignando de forma estricta unidades y choferes a sus respectivas terminales o sucursales físicas.
- Proveer un mecanismo ágil, seguro e interactivo para la comercialización de boletos a pasajeros mediante el uso de una Cartera Digital transaccional.
- Gestionar la cotización, programación y liquidación de viajes privados especiales a la medida solicitados por los clientes.
- Automatizar el cálculo de la depreciación vehicular por kilómetro recorrido y la auditoría de costos directos (combustible, repuestos y mano de obra en taller).
- Generar reportes financieros, operativos y comerciales con posibilidad de exportación estructurada en formato HTML.

Tecnologías y Herramientas Implementadas:

Para la construcción y puesta en marcha del proyecto se seleccionó un conjunto de tecnologías robustas y de alto rendimiento:

- **Java Development Kit (JDK 17/21):** Plataforma de desarrollo base orientada a objetos para la compilación y ejecución de lógica transaccional sobre la Máquina Virtual de Java (JVM).
- **Jakarta Servlets (versión 5.0+ / 6.0):** Componentes del lado del servidor que actúan como controladores HTTP encargados de interceptar peticiones GET/POST, procesar lógica de negocio, validar sesiones de usuario y gestionar el enrutamiento hacia las vistas.
- **JavaServer Pages (JSP) y JSTL:** Tecnología de presentación dinámica que permite generar interfaces web responsivas renderizadas en el servidor, consumiendo datos estructurados inyectados por los Servlets mediante el alcance de petición (request scope).
- **MySQL Server 8.0:** Sistema de Gestión de Bases de Datos Relacionales (RDBMS) implementado para la persistencia transaccional de los datos bajo el motor InnoDB.
- **JDBC (Java Database Connectivity):** API estándar que facilita la comunicación nativa entre los DAOs y el motor MySQL mediante PreparedStatement y control manual de transacciones ACID.
- **Apache Maven:** Herramienta de gestión de dependencias y automatización del ciclo de vida del proyecto, encargada de la compilación, vinculación del driver JDBC (mysql-connector-j) y empaquetado en artefacto WAR.
- **Apache Tomcat 10+:** Servidor web y contenedor de servlets que aloja y despacha la aplicación web.

- **HTML5, CSS3 y JavaScript:** Tecnologías del lado del cliente para maquetación moderna, diseño de paneles de control y mapas interactivos de selección de asientos.

II. Arquitectura de Software

El proyecto se diseñó bajo el patrón de arquitectura Modelo-Vista-Controlador (MVC) multicapa, garantizando una clara separación de responsabilidades, alta cohesión y bajo acoplamiento entre los módulos del sistema.

a) Capa de Presentación (Vistas - JSP): Comprende las páginas web (.jsp) que interactúan con el usuario final. Estas interfaces no contienen lógica de base de datos ni procesamiento de negocio; su función se limita a renderizar los formularios, tablas dinámicas y componentes gráficos, capturando eventos y reenviándolos a los controladores.

b) Capa de Control (Controladores - Servlets): Clases que heredan de `HttpServlet` y se encuentran mapeadas con anotaciones `@WebServlet`. Se encargan de validar los parámetros de entrada de las peticiones HTTP, controlar las sesiones de usuario (`HttpSession`), invocar a los métodos correspondientes en la capa DAO y redirigir el flujo mediante `RequestDispatcher`.

c) Capa de Acceso a Datos (DAOs): Clases especializadas (`Data Access Object`) que encapsulan la totalidad del código SQL. Ejecutan operaciones CRUD, sentencias preparadas contra inyecciones SQL y transacciones financieras complejas.

d) Capa de Modelos: Clases puras de Java que modelan fielmente las tablas de la base de datos y los objetos de transferencia de datos (DTOs) para reportes. Aplican encapsulamiento estricto con modificadores de acceso `private` y métodos `getters/setters`.

e) Capa de Infraestructura y Conexión (dbconnection): Módulo responsable de gestionar el ciclo de vida de las conexiones a la base de datos MySQL mediante el patrón Singleton y manejo de excepciones robusto.

Manejo de sesiones y seguridad

La autenticación y autorización se gestiona mediante sesiones seguras de HTTP (`HttpSession`). Al iniciar sesión exitosamente, el servlet almacena el identificador del usuario, su nombre y su `RolUsuario` ('ADMINISTRADOR_SISTEMA', 'ADMINISTRADOR_SUCURSAL', 'CLIENTE'). En cada petición a rutas protegidas, los servlets comprueban la existencia y vigencia de la sesión, impidiendo accesos no autorizados y forzando la redirección al formulario de login en caso de vencimiento.

III. Mapeo Físico de la Base de Datos

La persistencia de la plataforma se sustenta en una base de datos relacional, denominada `code_n_bugs`.

```
CREATE DATABASE IF NOT EXISTS code_n_bugs;
USE code_n_bugs;

CREATE TABLE Usuarios (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    dpi VARCHAR(13) NOT NULL UNIQUE,
    password VARCHAR(100) NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    nit VARCHAR(13) NOT NULL UNIQUE,
    telefono VARCHAR(8) NOT NULL UNIQUE,
    direccion VARCHAR(200),
    estado BOOLEAN DEFAULT TRUE,
    rol ENUM('ADMINISTRADOR_SISTEMA', 'ADMINISTRADOR_SUCURSAL', 'CLIENTE') NOT NULL
);

CREATE TABLE Sucursales (
    id_sucursal INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    direccion VARCHAR(200) NOT NULL,
    telefono VARCHAR(15) NOT NULL UNIQUE,
    latitud DOUBLE DEFAULT 14.83472,
    longitud DOUBLE DEFAULT -91.51805,
    tarifa_base_hora DECIMAL (10,2) NOT NULL DEFAULT 75.00,
    tarifa_pasajero DECIMAL (10,2) NOT NULL DEFAULT 20.00
);

CREATE TABLE Admin_Sucursal (
    id_asignacion INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL,
    id_sucursal INT NOT NULL,
    FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario),
    FOREIGN KEY (id_sucursal) REFERENCES Sucursales(id_sucursal)
);
```

```

);

CREATE TABLE Configuracion_Sistema (
    id_configuracion INT AUTO_INCREMENT PRIMARY KEY,
    depreciacion_por_km DECIMAL(10,2) NOT NULL
);

CREATE TABLE Choferes (
    id_chofer INT AUTO_INCREMENT PRIMARY KEY,
    id_sucursal INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    foto MEDIUMBLOB,
    num_licencia VARCHAR(50) NOT NULL UNIQUE,
    tipo_licencia ENUM('TIPO_A', 'TIPO_B', 'TIPO_C', 'TIPO_D', 'TIPO_E') NOT NULL,
    fecha_vencimiento_licencia DATE NOT NULL,
    telefono VARCHAR(15) NOT NULL UNIQUE,
    salario_base_por_viaje DECIMAL(10,2) NOT NULL,
    estado BOOLEAN DEFAULT TRUE,
    estado_operativo ENUM('DISPONIBLE', 'EN_RUTA', 'DESCANSO') NOT NULL,
    FOREIGN KEY (id_sucursal) REFERENCES Sucursales(id_sucursal)
);

CREATE TABLE Buses (
    id_bus INT AUTO_INCREMENT PRIMARY KEY,
    id_sucursal INT NOT NULL,
    id_chofer INT NULL,
    foto MEDIUMBLOB,
    placa VARCHAR(7) NOT NULL UNIQUE,
    marca VARCHAR(50) NOT NULL,
    modelo VARCHAR(50) NOT NULL,
    anio_fabricacion INT NOT NULL,

```

```

    capacidad INT NOT NULL,
    estado_operativo ENUM('DISPONIBLE', 'EN_RUTA', 'EN_MANTENIMIENTO', 'INACTIVO') NOT NULL,
    kilometraje_actual DECIMAL(10,2) NOT NULL,
    estado BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (id_sucursal) REFERENCES Sucursales(id_sucursal),
    FOREIGN KEY (id_chofer) REFERENCES Choferes(id_chofer)
);

CREATE TABLE Mantenimiento (
    id_mantenimiento INT AUTO_INCREMENT PRIMARY KEY,
    id_bus INT NOT NULL,
    fecha_mantenimiento DATE NOT NULL,
    monto_mano_obra DECIMAL(10,2) NOT NULL,
    monto_repuestos DECIMAL(10,2) NOT NULL,
    descripcion VARCHAR(260),
    FOREIGN KEY (id_bus) REFERENCES Buses(id_bus)
);

CREATE TABLE Rutas (
    id_ruta INT AUTO_INCREMENT PRIMARY KEY,
    id_origen INT NOT NULL,
    id_destino INT NOT NULL,
    distancia_km DECIMAL(10,2) NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (id_origen) REFERENCES Sucursales(id_sucursal),
    FOREIGN KEY (id_destino) REFERENCES Sucursales(id_sucursal)
);

CREATE TABLE Viajes (
    id_viaje INT AUTO_INCREMENT PRIMARY KEY,
    estado_viaje ENUM('PROGRAMADO', 'EN_CURSO', 'FINALIZADO', 'CANCELADO') DEFAULT 'PROGRAMADO',
    id_bus INT NOT NULL,

```

```

CREATE TABLE Viajes (
    id_viaje INT AUTO_INCREMENT PRIMARY KEY,
    estado_viaje ENUM('PROGRAMADO', 'EN_CURSO', 'FINALIZADO', 'CANCELADO') DEFAULT 'PROGRAMADO',
    id_bus INT NOT NULL,
    id_chofer INT NOT NULL,
    id_ruta INT NOT NULL,
    fecha_hora_salida_estimada DATETIME NOT NULL,
    fecha_hora_llegada_estimada DATETIME NOT NULL,
    fecha_hora_salida_real DATETIME NULL,
    fecha_hora_llegada_real DATETIME NULL,
    kilometraje_salida DECIMAL(10,2) NULL,
    kilometraje_llegada DECIMAL(10,2) NULL,
    gasto_combustible DECIMAL(10,2) NULL,
    FOREIGN KEY (id_bus) REFERENCES Buses(id_bus),
    FOREIGN KEY (id_chofer) REFERENCES Choferes(id_chofer),
    FOREIGN KEY (id_ruta) REFERENCES Rutas(id_ruta)
);

CREATE TABLE Viajes_Privados (
    id_viaje_privado INT AUTO_INCREMENT PRIMARY KEY,
    id_cliente INT NOT NULL,
    id_sucursal INT NOT NULL,
    origen VARCHAR(150) NOT NULL,
    destino VARCHAR(150) NOT NULL,
    cantidad_pasajeros INT NOT NULL,
    precio DECIMAL(10,2) NULL,
    estado ENUM('PENDIENTE', 'COTIZADA', 'PAGADA', 'EN_CURSO', 'FINALIZADO', 'CANCELADO') DEFAULT 'PENDIENTE',

    -- Se llenan cuando el admin aprueba y programa
    id_bus INT NULL,
    id_chofer INT NULL,

    fecha_hora_salida_estimada DATETIME NOT NULL,

```

```

    precio DECIMAL(10,2) NULL,
    estado ENUM('PENDIENTE', 'COTIZADA', 'PAGADA', 'EN_CURSO', 'FINALIZADO', 'CANCELADO') DEFAULT 'PENDIENTE',

    -- Se llenan cuando el admin aprueba y programa
    id_bus INT NULL,
    id_chofer INT NULL,

    fecha_hora_salida_estimada DATETIME NOT NULL,
    fecha_hora_llegada_estimada DATETIME NULL,
    fecha_hora_salida_real DATETIME NULL,
    fecha_hora_llegada_real DATETIME NULL,
    kilometraje_salida DECIMAL(10,2) NULL,
    kilometraje_llegada DECIMAL(10,2) NULL,
    gasto_combustible DECIMAL(10,2) NULL,

    FOREIGN KEY (id_cliente) REFERENCES Usuarios(id_usuario),
    FOREIGN KEY (id_sucursal) REFERENCES Sucursales(id_sucursal),
    FOREIGN KEY (id_bus) REFERENCES Buses(id_bus),
    FOREIGN KEY (id_chofer) REFERENCES Choferes(id_chofer)
);

```

```

CREATE TABLE Boletos (
    id_boleto INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL,
    id_viaje INT NOT NULL,
    numero_asiento INT NOT NULL,
    precio_pagado DECIMAL(10,2) NOT NULL,
    fecha_pago DATETIME NOT NULL,
    FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario),
    FOREIGN KEY (id_viaje) REFERENCES Viajes(id_viaje)
);

```

```

CREATE TABLE Cartera (
    id_cartera INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL UNIQUE,
    cantidad_dinero DECIMAL(10,2) DEFAULT 0.00,
    FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario)
);

CREATE TABLE Transacciones (
    id_transaccion INT AUTO_INCREMENT PRIMARY KEY,
    id_cartera INT NOT NULL,
    monto DECIMAL(10,2) NOT NULL,
    tipo ENUM('RECARGA', 'PAGO_BOLETO', 'PAGO_ALQUILER') NOT NULL,
    fecha_hora DATETIME NOT NULL,
    descripcion VARCHAR(150),
    FOREIGN KEY (id_cartera) REFERENCES Cartera(id_cartera)
);

```

IV. Métodos Centrales y Lógica

A continuación se describen los procedimientos transaccionales más relevantes implementados en la capa de acceso a datos y controladores:

- **Compra Transaccional de Boletos(BoletoDAO):** La compra de un boleto involucra múltiples escrituras que deben ejecutarse como una unidad atómica para prevenir la sobreventa de asientos y garantizar la coherencia contable del saldo.
- **Verificación de disponibilidad operativa de buses y choferes:** Para evitar que un autobús o un conductor sea asignado simultáneamente a múltiples viajes en conflicto de horario, el sistema ejecuta validaciones temporales en ViajeDAO y ViajePrivadoDAO.
- **Cotización de viajes privados:** Los viajes privados no tienen un precio fijo de ruta; su tarifa se calcula en base a los parámetros configurados en la sucursal asignada:

$$\text{Tarifa Total} = (\text{Horas de Servicio Estimadas} \times \text{Tarifa Base por Hora}) + (\text{Cantidad de Pasajeros} \times \text{Tarifa por Pasajero})$$

El Administrador de Sucursal puede ajustar el valor propuesto antes de enviar la cotización al cliente para su aprobación y pago.
- **Asentamiento de cierre de viaje y depreciación vehicular:** Al arribar el vehículo a su destino, el chofer ingresa el odómetro de llegada y el gasto en combustible diésel. El sistema realiza automáticamente las siguientes acciones contables y logísticas:
 1. Calcula la distancia recorrida real: $\text{Distancia} = \text{Kilometraje Llegada} - \text{Kilometraje Salida}$.
 2. Actualiza el odómetro maestro del autobús en la tabla Buses.
 3. Obtiene el parámetro de depreciación vigente desde Configuracion_Sistema.
 4. Asienta el costo de depreciación del viaje: $\text{Costo Depreciación} = \text{Distancia} \times \text{Depreciación por Km}$.
 5. Restablece el estado operativo del bus y del chofer a 'DISPONIBLE'.

- **Motor de reportería financiera y operativa (ReportesDAO):** Es la clase encargada de hacer las diferentes consultas a la base de datos para extraer los datos y posteriormente armar el reporte necesario.

V. Inicio y Ejecución del Programa

Iniciar el servicio de MySQL Server en el puerto estándar 3306.

Abrir una sesión en MySQL Workbench o la terminal de comandos de MySQL.

Verificar que la base de datos 'code_n_bugs' y sus 13 tablas se hayan creado satisfactoriamente.

Localizar el archivo DBConnection.java en el paquete dbconnection y ajustar las constantes según el entorno.

Abrir una terminal en la raíz del proyecto donde reside el archivo 'pom.xml' y ejecutar el comando de construcción.

Maven resolverá las dependencias necesarias y creará el archivo ejecutable web 'proyecto1.war' dentro del directorio '/target/'.

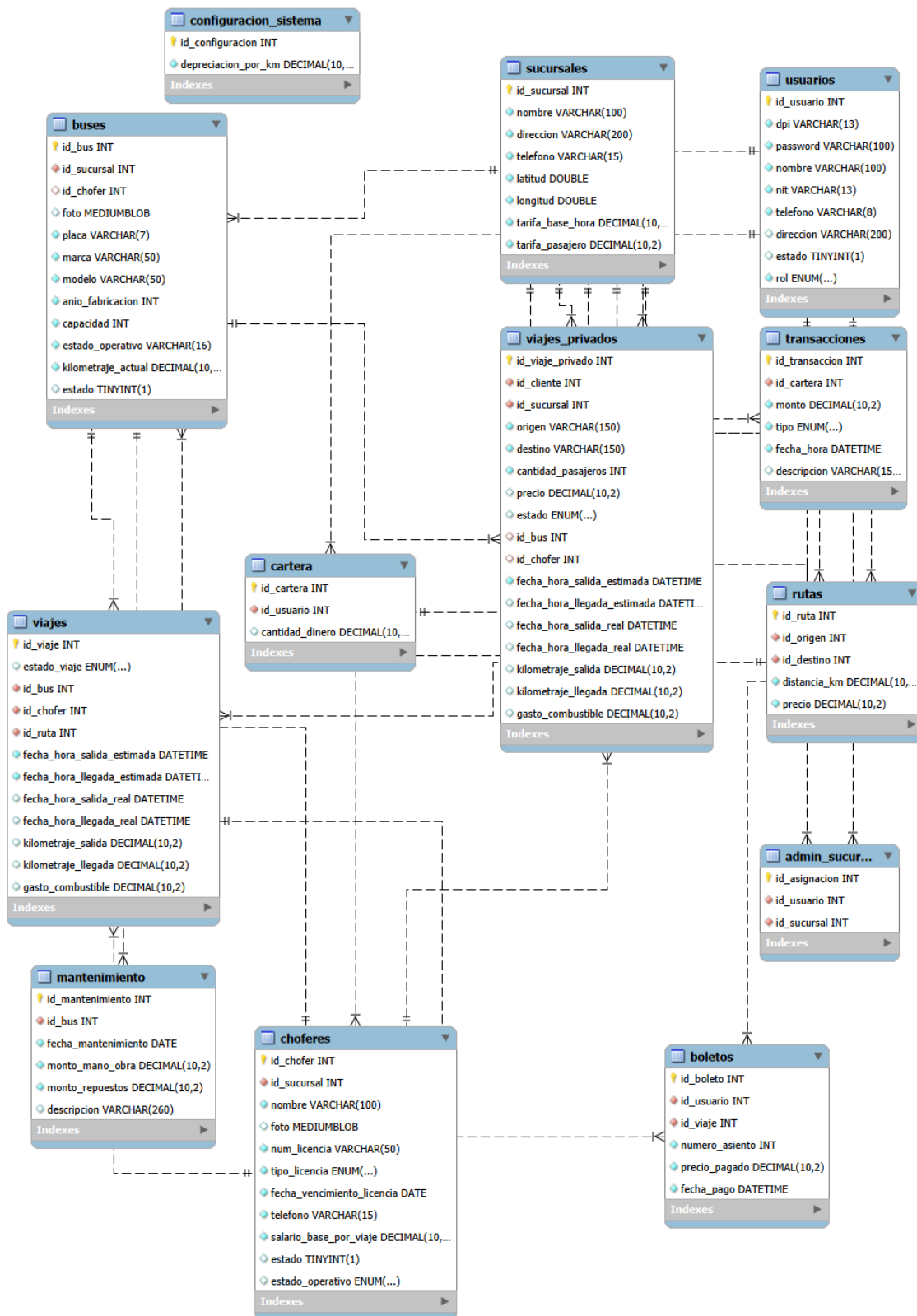
Luego de haber hecho eso, seguir los siguientes pasos:

1. Copiar el archivo 'proyecto1.war' al directorio 'webapps/' de la instalación de Apache Tomcat.
2. Iniciar el servidor.
3. Tomcat desplegará el archivo automáticamente extrayendo la aplicación web.
4. Abrir cualquier navegador web y navegar a la URL principal:

<http://localhost:8080/proyecto1/>

1. Diagrama Entidad - Relación (E/R)

1. Diagrama Entidad - Relación (E/R)



2. Diagrama de Tablas

CONFIGURACION SISTEMA											
id_configuracion (FK)	depreciacion_por_km										
SUCURSALES											
id_sucursal (FK)	nombre	direccion	telefono	latitud	longitud	tarifa_base_hora	tarifa_pasajero				
USUARIOS											
id_usuario (FK)	dpi	password	nombre	nit	telefono	direccion	estado	rol			
CHOFERES											
id_chofer (FK)	id_sucursal (FK)	nombre	foto	num_licencia	tipo_licencia	fecha_vencimiento_licencia	telefono	salario_base_por_viaje	estado	estado_operativo	
BUSES											
id_bus (FK)	id_sucursal (FK)	id_chofer (FK)	foto	placa	marca	modelo	año_fabricacion	capacidad	estado_operativo	kilometraje_actual	estado